Refactoring.to eclipse collections

Making Your Java Streams
Leaner, Meaner, and Cleaner

Donald Raab, Vladimir Zakharov

dev2next • Colorado Springs, CO • September 29 - October 2, 2025

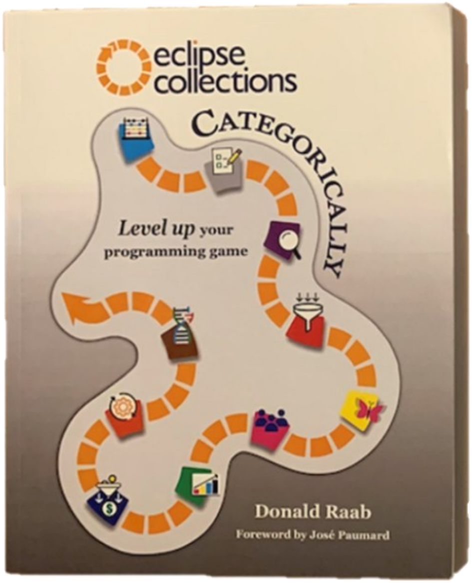
Introduction

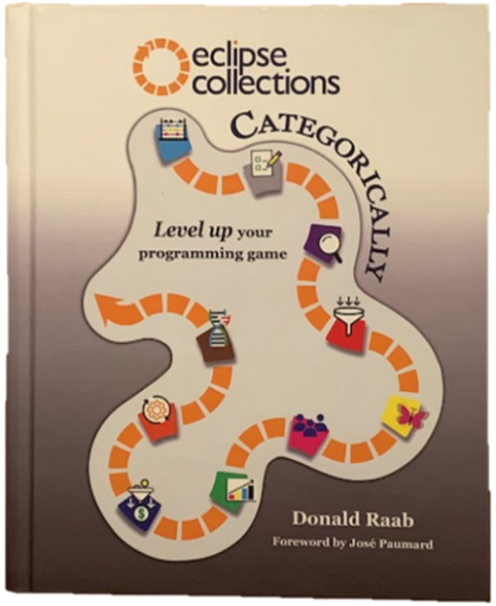
- What is Eclipse Collections?
  - Feature rich, memory efficient Java Collections framework
- History
  - Eclipse Collections started off as an internal collections framework named Caramel at Goldman Sachs in 2004

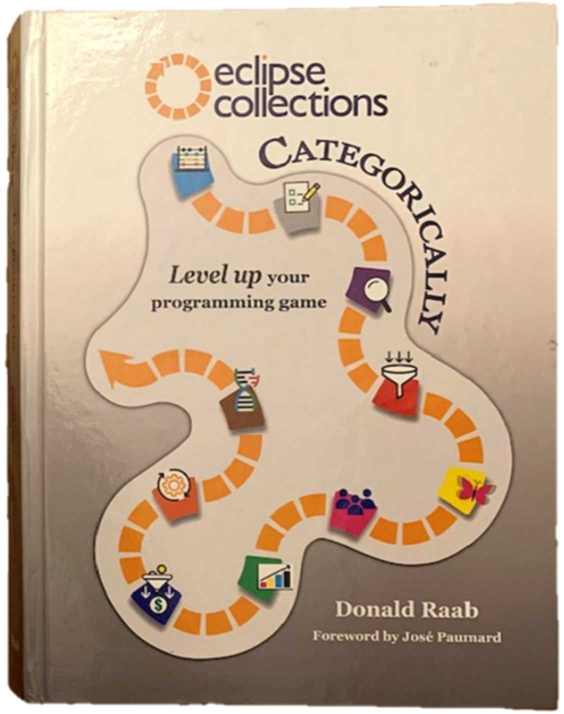
  - In 2012, it was open sourced to GitHub as a project called [GS Collections]
  - GS Collections was migrated to the Eclipse Foundation, re-branded as [Eclipse Collections] in 2015
  - Eclipse Collections 12.0 (Java 11+) & 13.0 (Java 17+) released in June 2025
- Learn Eclipse Collections with [Kata]
- Eclipse Collections is [open for contributions]!

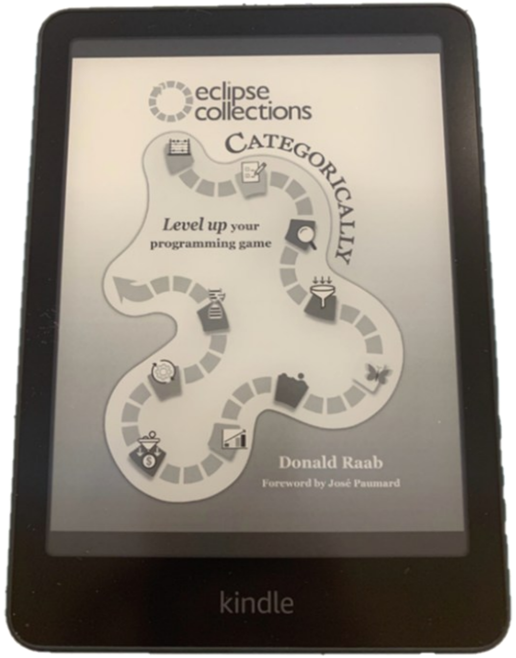
Book: Eclipse Collections Categorically

- Paperback, Hardcover(x2), and eBook Versions Available
- [Amazon](#) / [Barnes & Noble](#)

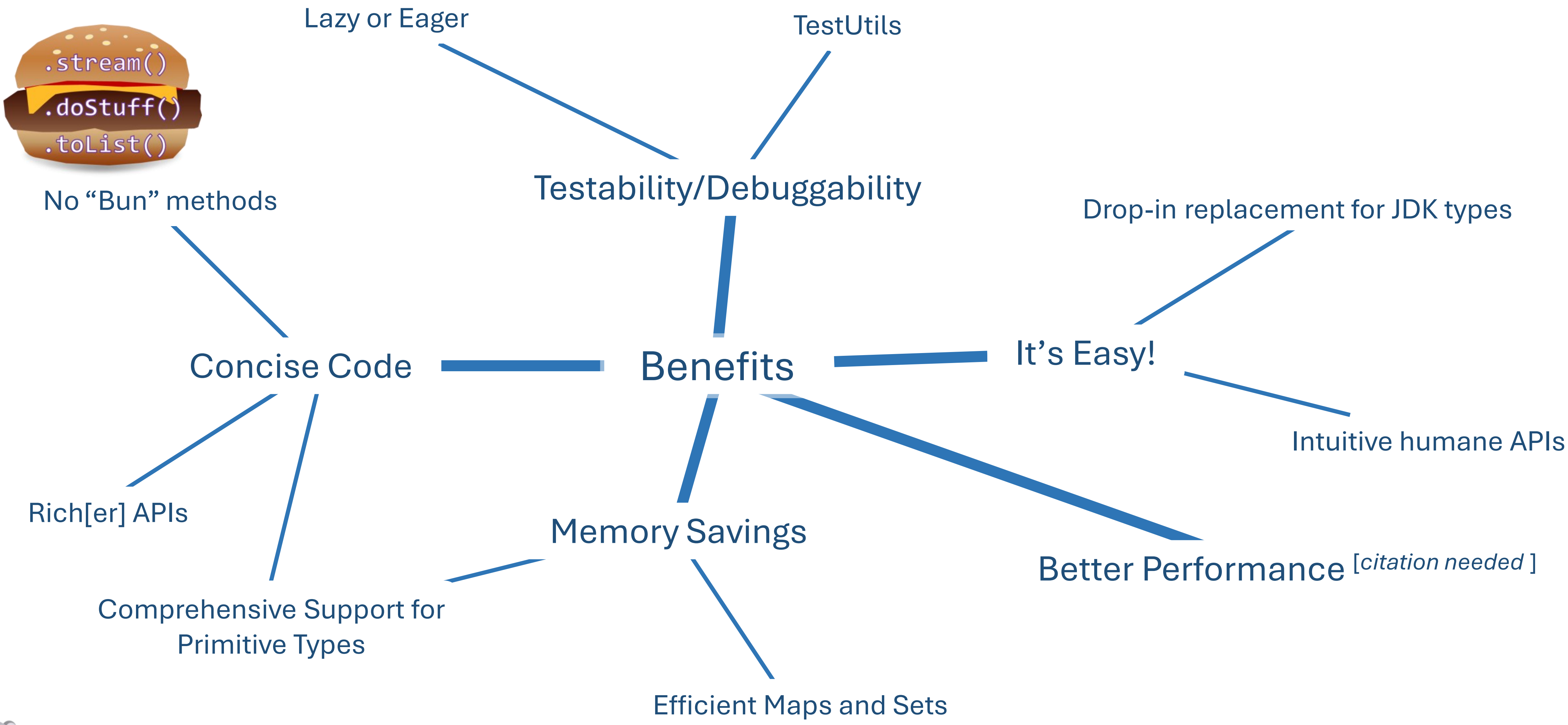




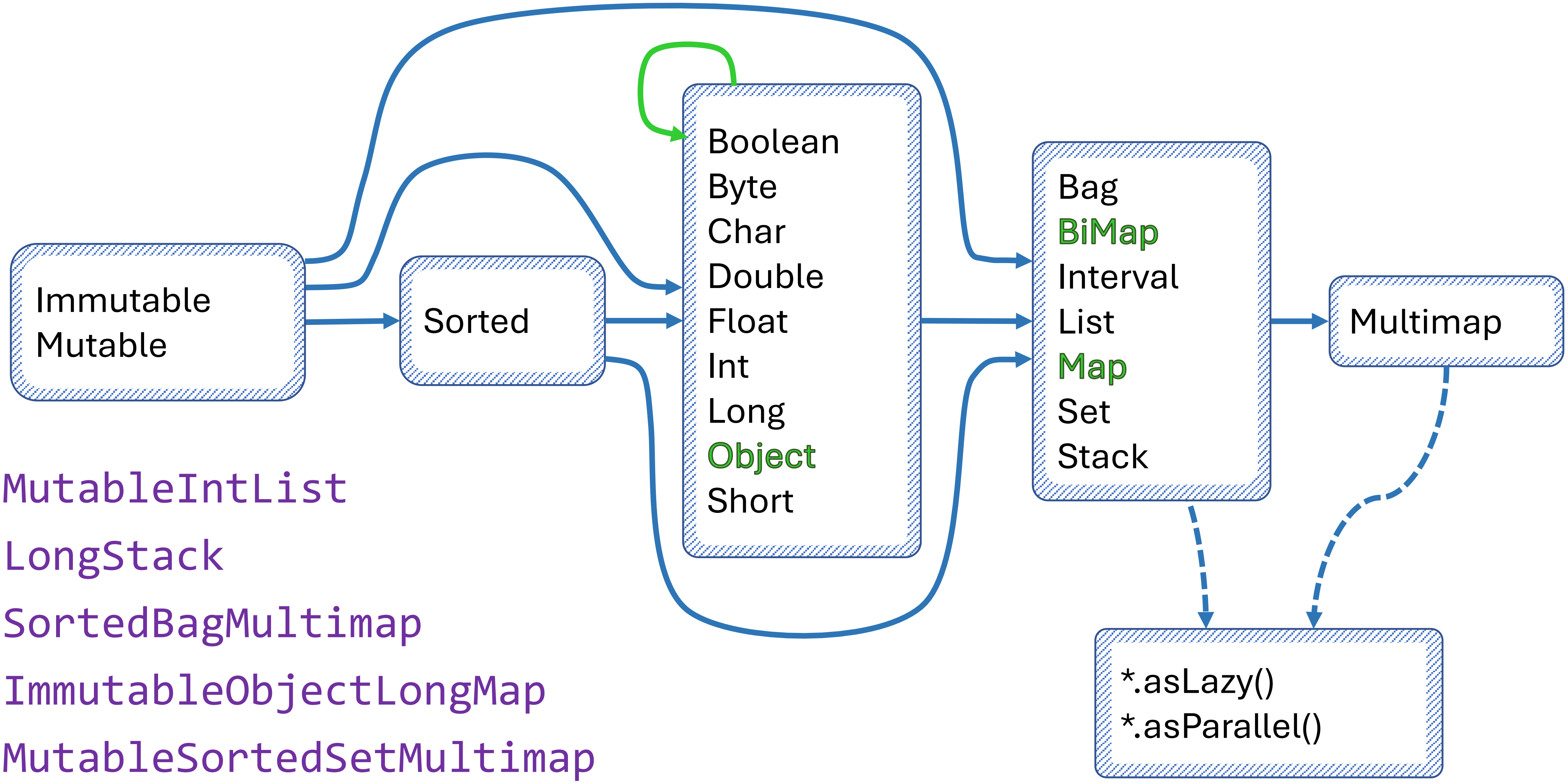




Why Refactor to EC?



Any Types You Need



MutableIntList

LongStack

SortedBagMultimap

ImmutableObjectLongMap

MutableSortedSetMultimap

Instantiate Them Using Factories

Primitive Type*	Container Type	Mutability	Multimap	Initialized	Lazy
Boolean	Bags	.mutable	.bag	.empty() | .of() | .with()	.asLazy()
Byte	BiMaps	.immutable	.list	.of(one) | .with(one)	
Char	Lists	.fixedSize	.set	...	
Double	Maps		.sortedSet	.of(one,…,ten)	
Float	Multimaps			.with(one,…,ten)	
Int	Sets			.of(... elements)	
Long	SortedBags			.with(... elements)	
Object	SortedMaps			.ofAll(Iterable)	
Short	SortedSets			.withAll(Iterable)	
	Stacks				

* *Pick two for maps*

ImmutableLongStack

LongStacks.***immutable***.with(1, 2, 3).asLazy()

LazyLongIterable

Methods by Category - Highlights

transform

collect[With]
collect[Boolean,Byte,Char,Double,Float,Int,Long,Short]
collectIf
collectKeysAndValues
collectValues
collectWithIndex
collectWithOccurrences
flatCollect

group

groupBy
groupByEach
groupByUniqueKey
sumBy[Double,Float,Int,Long]
sumOf[Double,Float,Int,Long]
aggregateBy
aggregateInPlaceBy

convert

toArray
toBag
toImmutable
toList
toMap
toReversed
toSet
toSortedBag[By]
toSortedList[By]
toSortedMap
toSortedSet
toSortedSet[By]
toStack
toString

filter

select[With]
selectByOccurrences
selectInstancesOf
reject[With]
partition[With]
partitionWhile

wrap

asLazy
asParallel
asReversed
asSynchronized
asUnmodifiable

find

detect[With]
detect[With]IfNone
detect[With]Optional
max[By]
min[By]

test

allSatisfy[With]
anySatisfy[With]
noneSatisfy[With]
notEmpty
isEmpty

Methods – Lots More…

Simulating Method Categories in Java

Let's Do It!

Memory: Methodology

Java Object Layout (JOL) - OpenJDK Code Tools Project

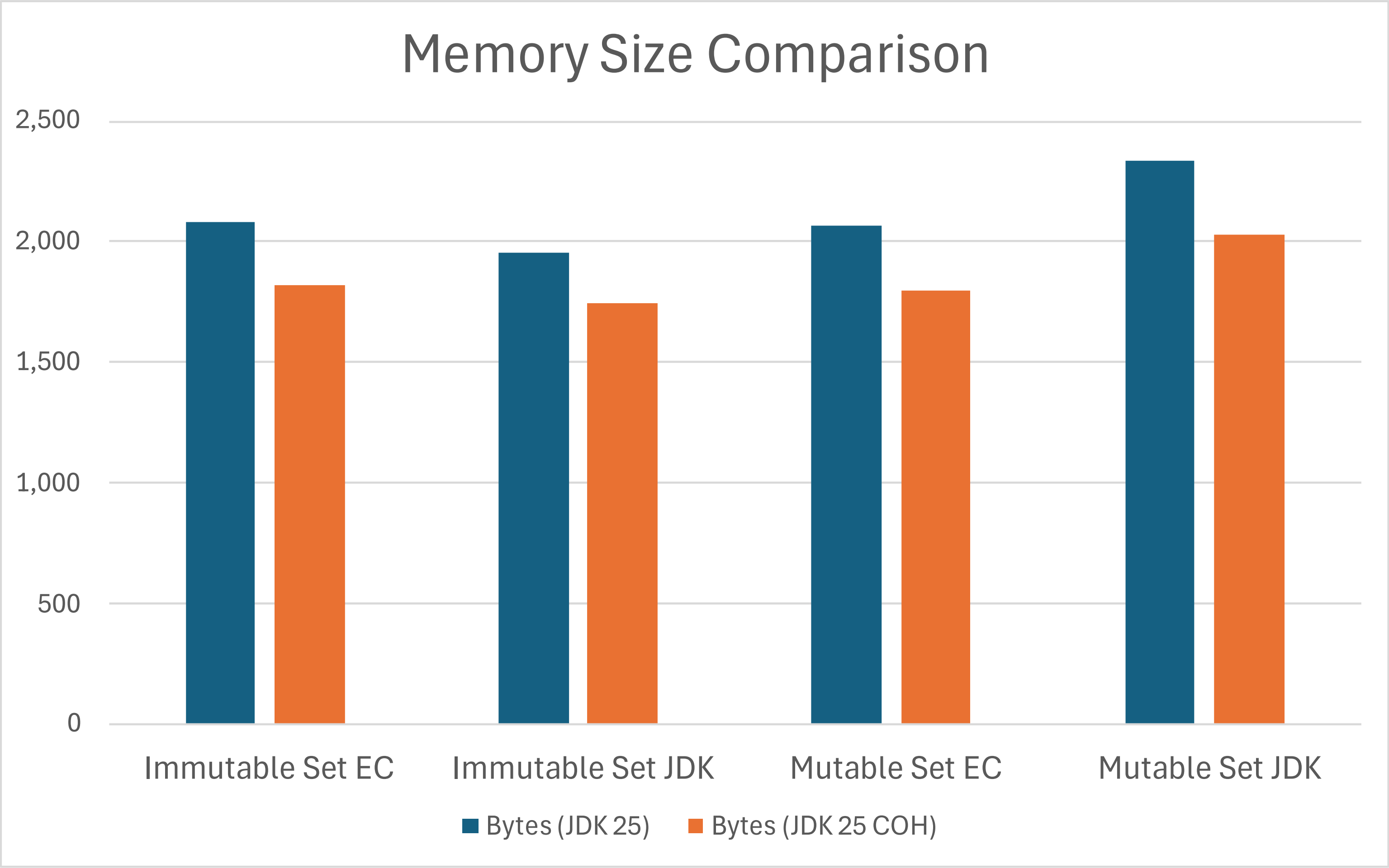
JOL (Java Object Layout) is the tiny toolbox to analyze object layout schemes in JVMs. These tools are using Unsafe, JVMTI, and Serviceability Agent (SA) heavily to decoder the actual object layout, footprint, and references. This makes JOL much more accurate than other tools relying on heap dumps, specification assumptions, etc.

```
<dependency>
    <groupId>org.openjdk.jol</groupId>
    <artifactId>jol-core</artifactId>
    <version>0.17</version>
</dependency>
```

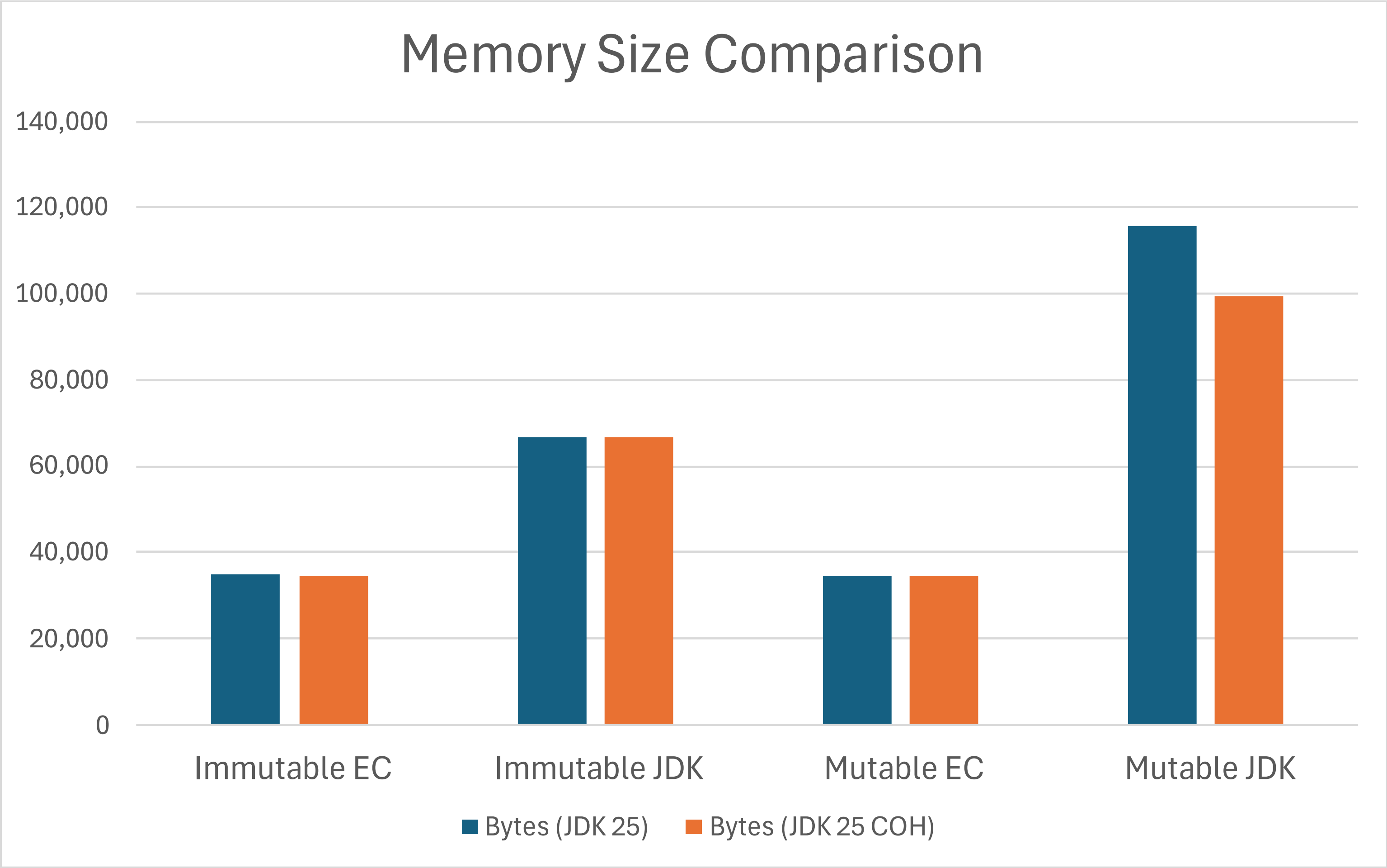
Further reading

- Baeldung: Memory Layout of Objects in Java
- StackOverflow: Does Java Object Layout work with Java Records?

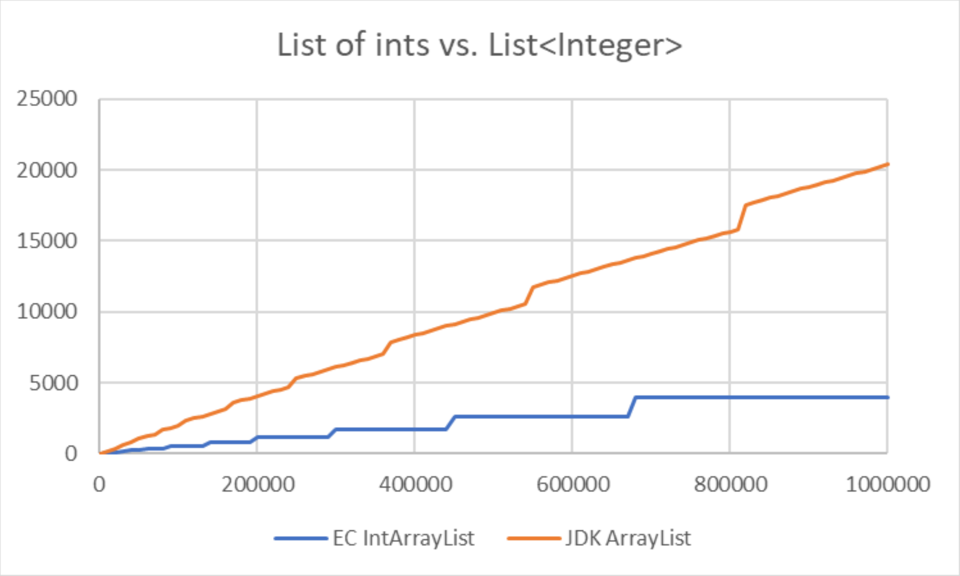
Memory: Generation Set

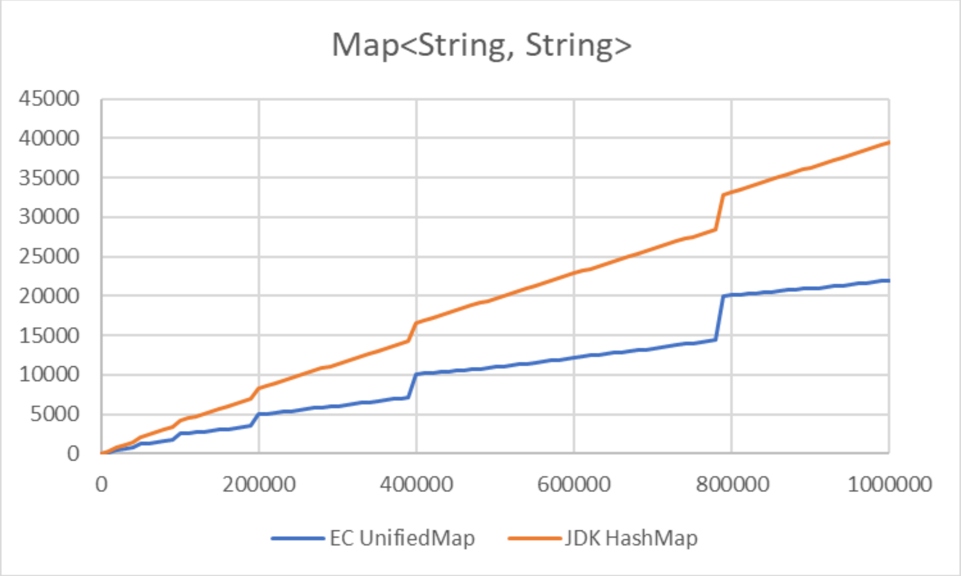


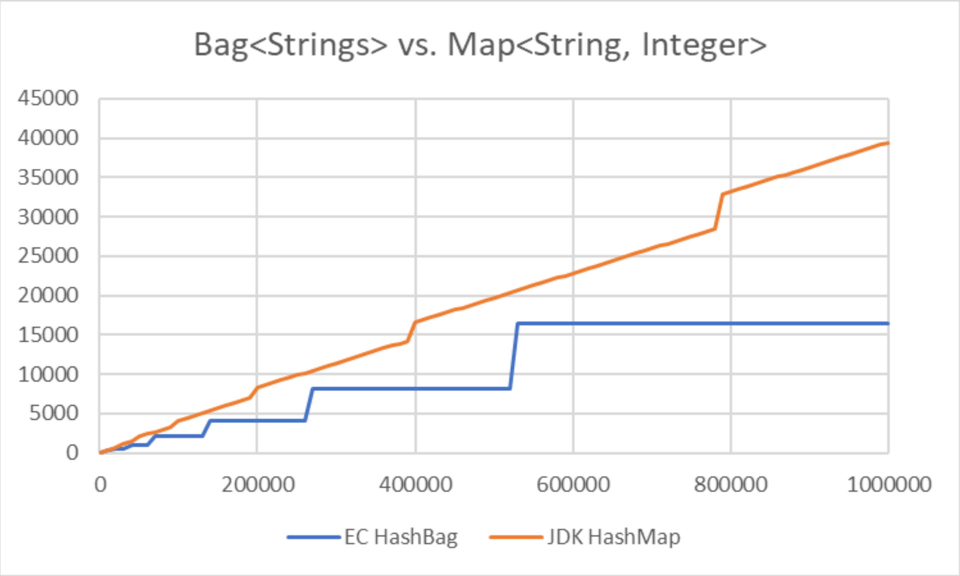
Memory: Year → Generation Map

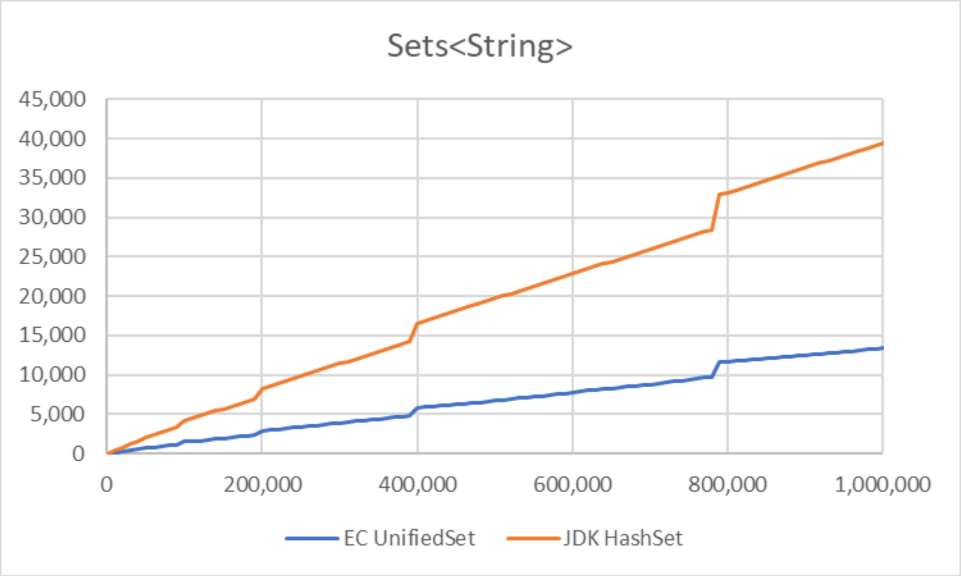


Memory Usage (Overhead in KB, Count)









Eclipse Collections – What's in It for Me?

Features	**Productivity**	**Performance**	**Thread Safety**	**Type Safety**
Rich functional API with good symmetry	✓			
Memory Efficiency		✓		
Optimized Eager API		✓		
Primitive Collections	✓	✓		✓
Immutable Collections		✓	✓	
Lazy and Parallel Lazy API		✓		
Multimap, Bag, BiMap, Pool, Interval		✓		✓
Hashing Strategy Data Structures	✓	✓		
Multi-reader Data Structures		✓	✓	
Mutable and Immutable Collection Factories	✓			

Example: DataFrame-EC

...built on the solid foundation of Eclipse Collections types

DataFrame

Customer	**Delivery Date**	**Donut**	**Quantity**	**Order Price**
Alice	7/10/2024	Old Fashioned	12	10.80
Alice	7/10/2024	Blueberry	2	2.50
Bob	7/10/2024	Old Fashioned	12	null
Alice	7/11/2024	Apple Cider	12	15.00
Alice	7/11/2024	Blueberry	2	2.50
Carol	7/11/2024	Old Fashioned	12	10.80
Dave	7/12/2024	Old Fashioned	12	10.80
Alice	7/12/2024	Jelly	12	15.00
Carol	7/12/2024	Blueberry	2	null
Bob	7/12/2024	Pumpkin Spice	null	0.75

Example: DataFrame-EC

…the foundation, and walls, and joints, and studs, and…

DataFrame

Customer	Delivery Date	Donut	Quantity		Order Price
Alice	7/10/2024	Old Fashioned	12	0	10.80
Alice	7/10/2024	Blueberry	2	0	2.50
Bob	7/10/2024	Old Fashioned	12	0	NaN
Alice	7/11/2024	Apple Cider	12	0	15.00
Alice	7/11/2024	Blueberry	2	0	2.50
Carol	7/11/2024	Old Fashioned	12	0	10.80
Dave	7/12/2024	Old Fashioned	12	0	10.80
Alice	7/12/2024	Jelly	12	0	15.00
Carol	7/12/2024	Blueberry	2	0	NaN
Bob	7/12/2024	Pumpkin Spice	-	1	0.75

MutableBooleanList bitmap

1
1
0
1
1
0
0
1
0
0

MutableMap<
    ListIterable<Object>,
    MutableIntList>
rowIndexByKey

["Alice"]	[0, 1, 3, 4, 7]
["Bob"]	[2, 9]
["Carol"]	[5, 8]
["Dave"]	[6]

Alice
Bob
Carol
Dave

Pool<String> pool

MutableIntList values

MutableBooleanList nullMap

MutableDoubleList values